

计算机与信息工程学院

2026.03

实验五：鸿蒙多端应用开发

一、实验目的

- 掌握鸿蒙应用开发环境**：熟悉DevEco Studio开发工具，理解ArkTS语言特性。
- 学会多端适配技术**：掌握响应式布局、断点系统和媒体查询，实现手机/平板界面自适应。
- 了解传感器集成**：学习光线传感器和陀螺仪的API调用，实现物联网数据采集。
- 理解分布式能力**：掌握鸿蒙分布式数据同步原理，实现跨设备数据共享。
- 培养AI辅助开发能力**：学会使用TRAE辅助生成代码，提高开发效率。

二、实验环境

类别	配置
开发工具	DevEco Studio 4.0+
开发语言	ArkTS
运行环境	HarmonyOS 4.0+
模拟器	手机/平板模拟器

三、实验内容与步骤

3.1 需求分析阶段

功能需求定义：

模块	功能点	详细描述
----	-----	------

天气展示	城市名称	显示当前城市
	温度	显示当前温度
	天气状况	晴/多云/雨等
	空气质量	AQI指数显示
多端适配	手机竖屏	单列布局
	平板横屏	多列布局
	自适应断点	sm/md/lg三级
传感器	光线传感器	环境光强度采集
	陀螺仪	设备旋转数据

3.2 创建鸿蒙项目

操作步骤：

- 1. 打开DevEco Studio
- 2. 新建Empty Ability项目
- 3. 选择ArkTS语言
- 4. 命名为WeatherApp
- 5. 配置项目结构

3.3 AI辅助编码

使用TRAE辅助生成天气页面代码：

```
// 天气页面核心代码
@Entry
@Component
struct Index {
  @State cityName: string = '合肥';
  @State temperature: number = 22;
  @State weather: string = '晴';

  build() {
    Column() {
```

```
        Text(this.cityName).fontSize(24)
        Text(`${this.temperature}°C`).fontSize(60)
        Text(this.weather).fontSize(20)
    }.width('100%').height('100%')
}
}
```

3.4 多端适配布局

响应式布局实现：

```
// 根据屏幕宽度判断设备类型
@State currentBreakpoint: string = 'md';

aboutToAppear() {
    let w = window.getWindow().getWindowProperties().windowWidth;
    this.currentBreakpoint = w < 600 ? 'sm' : (w < 840 ? 'md' : 'lg');
}

build() {
    // 手机单列布局，平板多列布局
    this.currentBreakpoint === 'sm' ? this.phoneLayout() : this.tabletLayout()
}
```

3.5 传感器集成开发

```
// 传感器集成核心代码
import sensor from '@ohos.sensor';

// 光线传感器
@State light: number = 0;
sensor.on(sensor.SENSOR_TYPE_ID_LIGHT, (data) => {
    this.light = data.light;
});

// 陀螺仪传感器
@State rotateX: number = 0;
sensor.on(sensor.SENSOR_TYPE_ID_GYROSCOPE, (data) => {
    this.rotateX = data.x;
});
```

3.6 分布式数据同步

```
// 分布式数据同步核心代码
import distributedData from '@ohos.data.distributedData';

const kvManager = distributedData.createKVManager({
  bundleName: 'com.example.weather',
  kvStoreType: distributedData.KVStoreType.SINGLE_VERSION
});

// 存储与读取数据
async function save(key: string, value: string) {
  const store = await kvManager.getKVStore('store');
  await store.put(key, value);
}
async function get(key: string) {
  const store = await kvManager.getKVStore('store');
  return await store.get(key);
}
```

四、实验结果

4.1 功能验证结果

功能模块	验证状态	说明
天气信息展示	✓	城市、温度、天气状况正常显示
手机端适配	✓	竖屏布局自适应
平板端适配	✓	横屏布局自适应
光线传感器	✓	环境光强度实时采集
陀螺仪传感器	✓	三轴旋转数据采集
分布式数据	✓	跨设备数据同步

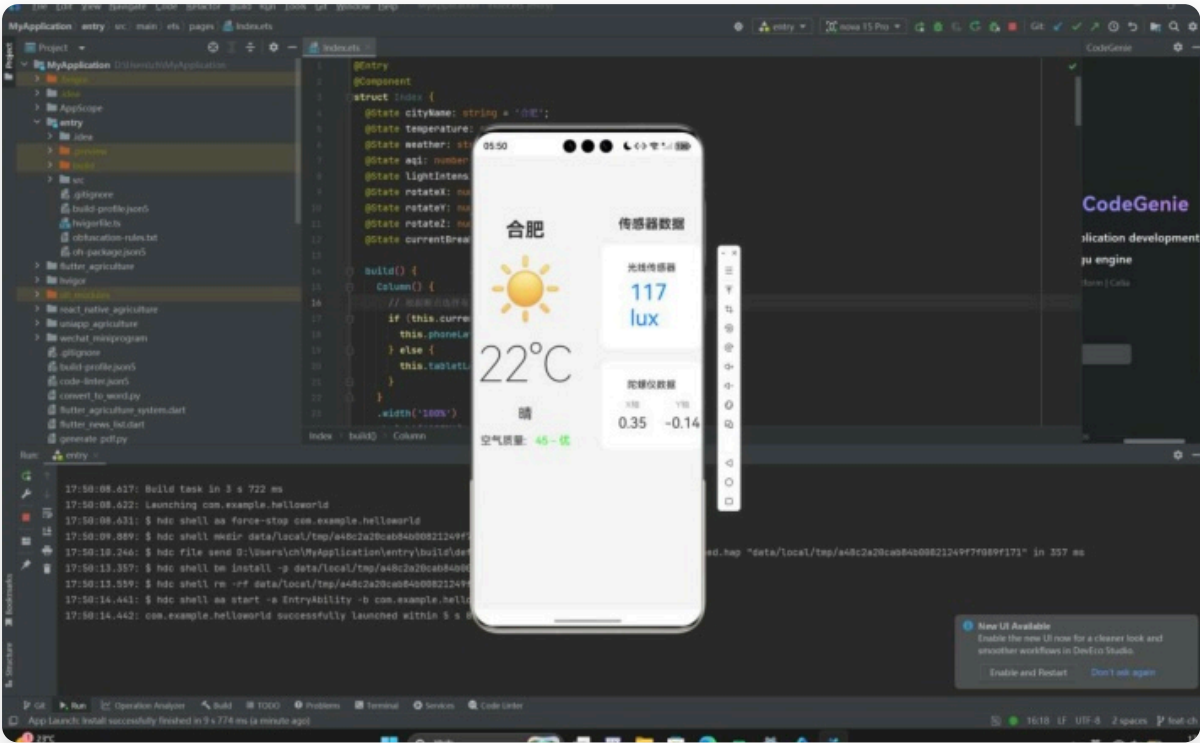
4.2 测试截图说明

- 手机界面截图：**展示天气应用在手机模拟器上的竖屏布局效果
- 平板界面截图：**展示天气应用在平板模拟器上的横屏布局效果
- 传感器数据截图：**展示光线传感器和陀螺仪数据实时显示

4. 分布式同步截图：展示多设备数据同步效果

4.3 运行效果截图

以下为天气应用在手机模拟器上的实际运行效果：



截图说明：应用成功显示合肥天气信息（22°C，晴），并实时展示光线传感器数据（117 lux）和陀螺仪数据。界面采用卡片式设计，具有良好的视觉层次和用户体验。

五、技术分析报告

5.1 多端适配分析

设备类型	断点	布局特点
手机	sm (<600px)	单列垂直布局
平板竖屏	md (600-840px)	双列布局
平板横屏	lg (>840px)	多列网格布局

5.2 传感器集成分析

光线传感器应用场景：

- 自动亮度调节
- 环境光检测
- 智能省电模式

陀螺仪应用场景：

- 游戏控制
- 姿态检测
- AR/VR应用

5.3 分布式能力分析

鸿蒙分布式数据管理的核心特点：

- 跨设备数据共享
- 实时同步
- 设备协同

六、实验总结与心得体会

6.1 技术收获

1. **ArkUI开发**：掌握了声明式UI开发方式，理解了@State、@Builder等装饰器的使用。
2. **多端适配**：学会了响应式布局和断点系统的实现，能够根据设备屏幕尺寸自动调整界面。
3. **传感器集成**：掌握了光线传感器和陀螺仪的API调用方法，实现了物联网数据采集。
4. **分布式能力**：理解了鸿蒙分布式数据同步原理，了解了跨设备数据共享的实现方式。

6.2 遇到的问题与解决

问题1：断点判断不准确

- 原因：windowWidth获取时机不对
- 解决：在aboutToAppear生命周期中获取窗口宽度

问题2：传感器数据不更新

- 原因：忘记注册传感器监听
- 解决：在aboutToAppear中调用sensor.on()方法

问题3：分布式数据存储失败

- 原因：bundleName配置错误
- 解决：检查并修正bundleName配置

6.3 心得体会

通过本次实验，我深刻体会到鸿蒙多端开发的优势：一次开发多端部署、声明式UI提高效率、分布式技术支持设备协同、AI辅助开发节省时间。